



**Department of Computer Science and Engineering**  
**Midterm Examination Fall 2025**  
**CSE 321: Operating Systems**

Duration: 1 Hour 30 Minutes

Total Marks: 30

Answer all the following questions.  
 Figures in the right margin indicate marks.

|       |     |      |
|-------|-----|------|
| Name: | ID: | Sec: |
|-------|-----|------|

1. a) Write the answer in the question paper. CA-E [1]

**CO1** A system is running five heavy machine-learning processes that are all CPU-bound. The OS frequently switches between these processes to share CPU time, even though each process needs long continuous CPU bursts.

**Explain** the effect this frequent switching will have on system performance.

**Answer:**

b) Write the outputs in the question paper. The output sequence should be exactly matched. CA-M [1\*3 =3]

Write the outputs of each print statement in one line.

```
main:
  int p = 10, q = 15, r = 20
  pid1 = fork()
  if (pid1 == 0)
    p = p * 3
    q = q / 5
    r = r + 4
    pid2 = fork()
    if (pid2 == 0)
      p = p + 3
      q = q - 3
      r = r / 8
      print("p:", p, " q:", q, " r:", r)
    else
      wait(NULL)
      print("p:", p, " q:", q, " r:", r)
  else
    wait(NULL)
    p = p - 7
    q = q + 13
    r = r * 2
    print("p:", p, " q:", q, " r:", r)
```

| Outputs |  |
|---------|--|
| I       |  |
| II      |  |
| III     |  |

**c) Write the answers in the question paper.**

Fariha has designed an operating system for her project, but has not implemented dual-mode operation.

**CA-E [1\*2  
=2]**

- I. **Explain** the problems that can be caused by user applications not implementing dual mode.
- II. **Explain** how dual-mode can prevent user applications from directly accessing system resources.

**Answer:**

|    |  |
|----|--|
| I  |  |
| II |  |

**2. a) Write the answer in the answer script.**

**CA-M [2]**

**CO1** Dipu has implemented two identical MLFQ schedulers. In the first implementation, new processes arrive at the queue having the highest priority level. In the second implementation, new processes arrive at the queue having the lowest priority level. Apart from the mentioned difference, all other operations of MLFQ are the same as regular MLFQs. A balanced number of I/O and CPU-bound processes arrive in the same order in both implementations of Dipu.

**Explain** how the different scheduling criteria would differ across both implementations.

**b) Write the answers in the answer script.****CA-E**

In an OS, **Multilevel Feedback Queue** functions as a scheduler. In the scheduler, the ready queue is divided into **three** levels. At every level, the round-robin algorithm has been employed, and certain constraints must be maintained, as listed below.

- **Q0:** time quantum = **3**, priority = **0**
- **Q1:** time quantum = **5**, priority = **1**
- **Q2:** time quantum = **7**, priority = **2**

Here, the lower priority value indicates a higher priority, and in the case of an I/O request, the scheduler will promote a process by one level upon completion of the I/O operation. The information on burst times, arrival times, and I/O times of three processes is given below:

| Process | Burst Time | Arrival Time | I/O Time                                      |
|---------|------------|--------------|-----------------------------------------------|
| P1      | 12         | 2            | 5s I/O [after total <b>7s</b> CPU allocation] |
| P2      | 9          | 0            | 3s I/O [after total <b>6s</b> CPU allocation] |
| P3      | 10         | 5            | N/A                                           |

- I. **Draw** a Gantt chart using the multilevel feedback queue scheduling algorithm, showing the states of the ready queues of different levels.
- II. **Calculate** the average waiting time and average turnaround time from the Gantt chart.

**[5]****[1+1  
=2]**

c) Write the answer in the question paper.

CA-D [2]

Kabbya is attempting to design a CPU scheduling mechanism based on the multilevel feedback queue scheduler. In his implementation, he aims to ensure that specific processes **always** have the highest priority, while other processes follow the regular promotion/demotion mechanism.

**Explain**, using proper logic, how Kabbya can implement this behavior in his operating system.

**Answer:**

|  |
|--|
|  |
|--|

3. a) Write the answer in the question paper.

CA-M [2]

CO2 A programmer attempts to use a semaphore to protect a shared counter **x**, as shown below:

|                                                                                |                                                                                |
|--------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| <pre>semaphore mutex = 1; int x = 0;</pre>                                     |                                                                                |
| <pre>// Thread 1 int temp = x; wait(mutex); x = temp + 1; signal(mutex);</pre> | <pre>// Thread 2 int temp = x; wait(mutex); x = temp + 1; signal(mutex);</pre> |

**Identify** the synchronization mistake in this code.

**Answer:**

|  |
|--|
|  |
|--|

**b) Write the answer in the question paper.****CA-E [1]**

Consider a CPU-bound process running on two different operating systems A and B, each running on a CPU with 8 cores. In OS A, the process runs with 8 user threads and 1 kernel thread, and in OS B, it runs with 16 user threads and 1 kernel thread.

**Explain** how the execution time of the process in OS B compares to the execution time of the process in OS A.

**Answer:**

**c) Write the answer in the answer script.****CA-D [2]**

**Write** the issues with proper logic that might arise if we use recursive functions in a multi-threaded program.

**d) Write the answer in the answer script.****CA-D**

The variable **counter** is a global variable that is shared by both **thread-1** and **thread-2**. The initial value of **counter** is 100, **temp1** and **temp2** are local variables. Consider **thread-1** and **thread-2** are run at the same time.

| Code run by thread-1                                                                                            | Code run by thread-2                                                                                            |
|-----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <pre>// unrelated statements temp1 = counter; temp1 = temp1 * 2; counter = temp1; // unrelated statements</pre> | <pre>// unrelated statements temp2 = counter; temp2 = temp2 / 4; counter = temp2; // unrelated statements</pre> |

- I. **Identify** what kind of problem can arise in this scenario and how it can be avoided.

**[2]**

- II. **Modify** the codes of both **thread-1** and **thread-2** above by utilizing **two binary semaphores** to ensure that the final value of **counter** is **greater** than its initial value, after both threads have finished their execution. You have to mention the initial values of both semaphores.

**[1.5+  
1.5=  
3]**

- e) Write the outputs in the question paper. The output sequence should be exactly matched. CA-M [1\*3=3]

```

value = 10
mutex m
semaphore s1
semaphore s2

function worker1(arg1):
    semaphore_wait(s1)
    mutex_lock(m)
    value = value * 3
    print("Thread 1 result ", value)
    mutex_unlock(m)
    semaphore_signal(s2)

function worker2(arg2):
    mutex_lock(m)
    value = value - 2
    print("Thread 2 result ", value)
    mutex_unlock(m)
    semaphore_signal(s1)

function worker3(arg3):
    semaphore_wait(s2)
    mutex_lock(m)
    value = value + 5
    print("Thread 3 result ", value)
    mutex_unlock(m)
    semaphore_signal(s2)

main():
    mutex_initialize(m, NULL)
    semaphore_initialize(s1, 0, 0)
    semaphore_initialize(s2, 0, 0)
    thread_create(t1, NULL, worker1, NULL)
    thread_create(t2, NULL, worker2, NULL)
    thread_create(t3, NULL, worker3, NULL)
    thread_join(t1, NULL)
    thread_join(t2, NULL)
    thread_join(t3, NULL)
    mutex_destroy(m)
    semaphore_destroy(s1)
    semaphore_destroy(s2)

```

| Outputs |  |
|---------|--|
| I       |  |
| II      |  |
| III     |  |